

Python lists and dictionaries are values that can contain multiple other values. To access these values, you use an *index operator*, which is composed of a pair of square brackets (`[]`) and an integer called an *index* to specify which value you want to access. Enter the following into the interactive shell to see how indexing works with lists:

```
>>> spam = ['cat', 'dog', 'moose']
>>> spam[0]
'cat'
>>> spam[-2]
'dog'
```

In this example, `0` is an index. The first index is `0`, not `1`, because Python (as most languages do) uses *zero-based indexing*. Languages that use one-based indexing are rare: Lua and R are the most predominant. Python also supports negative indexes, where `-1` refers to the last item in a list, `-2` refers to the second-to-last item, and so on. You can think of a negative index `spam[-n]` as being the same as `spam[len(spam) - n]`.

NOTE

Computer scientist and singer-songwriter Stan Kelly-Bootle once joked, “Should array indices start at 0 or 1? My compromise of 0.5 was rejected without, I thought, proper consideration.”

You can also use the index operator on a list literal, although all those square brackets can look confusing and unnecessary in real-world code:

```
>>> ['cat', 'dog', 'moose'][2]
'moose'
```

Indexing can also be used for values other than lists, such as on a string to obtain individual characters:

```
>>> 'Hello, world'[0]
'H'
```

Python dictionaries are organized into *key-value pairs*: